

Q2. Write a thread-safe singleton class (why private constructor + static field + static method).

A thread-safe singleton class ensures that only one instance exists across all threads. It uses a private constructor to prevent direct instantiation, a private static field to hold the single instance, and a public static method (such as `getInstance`) to provide global access. Thread safety can be achieved with synchronized access, double-checked locking, or a static inner class.

Q3. How to create a new thread? Why consider thread pools?

A new thread can be created by extending `Thread` and overriding `run()`, or by implementing `Runnable` or `Callable` and passing it to a `Thread`. Thread pools are preferred in real-world applications because they reuse a fixed number of threads, reduce creation overhead, and provide better control over concurrency.

Q4. Difference between `Runnable` and `Callable`?

`Runnable` represents a task that returns no result and cannot throw checked exceptions, while `Callable` represents a task that returns a result and can throw checked exceptions. `Callable` is used with `ExecutorService` to obtain a `Future` representing the result.

Q5. Difference between `t.start()` and `t.run()`?

Calling `t.start()` creates a new thread and executes the `run()` method in that new thread. Calling `t.run()` directly executes `run()` in the current thread without starting a new thread.

Q6. Which is better: implementing `Runnable` or extending `Thread`?

Implementing `Runnable` is usually better because it allows a class to extend another superclass and cleanly separates the task from the thread execution mechanism. Extending `Thread` tightly couples the task to the `Thread` class and limits inheritance.

Q7. What are the main thread states?

Typical thread states include `NEW` (created but not started), `RUNNABLE` (ready or running), `BLOCKED` (waiting for a monitor lock), `WAITING` (waiting indefinitely for another thread), `TIMED_WAITING` (waiting with a timeout), and `TERMINATED` (finished execution).

Q8. What is deadlock and how to resolve it?

Deadlock occurs when two or more threads are waiting on each other to release resources, causing all of them to block forever. It can be prevented by enforcing a strict lock acquisition order, using `tryLock` with timeout, reducing lock scope, or using higher-level concurrency utilities.

Q9. How do threads communicate with each other?

Threads communicate mainly through shared variables and synchronization mechanisms such as `wait()`, `notify()`, and `notifyAll()` on an object monitor. Higher-level constructs like `BlockingQueue`, `Exchanger`, and `CountDownLatch` also support safe communication and coordination.

Q10. Difference between class lock and object lock?

An object lock is associated with a specific instance and is acquired by synchronizing on that instance or using a synchronized instance method. A class lock is associated with the `Class` object and is acquired by synchronizing on the `Class` or using a static synchronized method. Class locks serialize access across all instances of that class.

Q11. What is join()?

join() is a Thread method that causes the current thread to wait until the target thread finishes its execution. It is commonly used to ensure that a worker thread completes before the main thread proceeds.

Q12. What is yield()?

yield() is a hint to the scheduler that the current thread is willing to pause and allow other threads of the same priority to run. It does not guarantee any specific scheduling behavior and should not be relied upon for correctness.

Q13. What is a ThreadPool? What types and what is a task queue?

A ThreadPool is a group of worker threads that execute submitted tasks. Common types include fixed thread pools, cached thread pools, single-thread executors, and scheduled thread pools. The task queue holds pending tasks waiting for available worker threads to execute them.

Q14. Which library is used to create ThreadPool and which interface defines its main functions?

The java.util.concurrent library is used to create thread pools. The Executor and ExecutorService interfaces define the main thread-pool operations such as execute, submit, shutdown, and invokeAll.

Q15. How do you submit a task to a ThreadPool?

A task can be submitted by calling execute(Runnable) or submit(Runnable/Callable) on an ExecutorService. submit returns a Future that can be used to check completion or retrieve the result for Callable tasks.

Q16. What are the advantages of ThreadPool?

Thread pools improve performance by reusing threads, limit the number of concurrent threads to avoid resource exhaustion, and provide centralized management of task submission, scheduling, and error handling.

Q17. Difference between shutdown() and shutdownNow()?

shutdown() initiates an orderly shutdown in which previously submitted tasks are executed but no new tasks are accepted. shutdownNow() attempts to stop all actively executing tasks and returns a list of tasks that were submitted but not yet executed.

Q18. What are Atomic classes? Types and usage example.

Atomic classes provide lock-free, thread-safe operations on single variables using low-level hardware support. Common types include AtomicInteger, AtomicLong, AtomicBoolean, and AtomicReference. For example, AtomicInteger can be used to implement a thread-safe counter with incrementAndGet() without explicit synchronization.

Q19. What are concurrent collections? Name some.

Concurrent collections are thread-safe collection implementations designed for high concurrency with minimal locking. Examples include ConcurrentHashMap, CopyOnWriteArrayList, ConcurrentLinkedQueue, and LinkedBlockingQueue.

Q20. What kinds of locks do you know and what are their advantages?

Common lock types include intrinsic locks (synchronized), ReentrantLock, ReadWriteLock, and StampedLock. ReentrantLock offers features such as fairness policies and tryLock with timeout. ReadWriteLock allows multiple readers but only one writer. StampedLock improves read performance with optimistic reads in highly concurrent scenarios.

Q21. What are Future and CompletableFuture? Main methods of CompletableFuture.

Future represents the result of an asynchronous computation but provides limited control and blocking get(). CompletableFuture extends this with a rich, non-blocking API for composing async tasks. Key methods include supplyAsync(), thenApply(), thenAccept(), thenCompose(), thenCombine(), exceptionally(), and allOf().